



Mòdul 487

Entorns de desenvolupament

Diagrama de classes i d'instàncies

- Mostra **l'estructura del sistema dissenyat a nivell de classes i interfícies**:
 - Quines classes componen el sistema?
 - Com són aquestes classes (tipus, atributs, ...)?
- Mostra les seves **característiques, restriccions i relacions**: associacions, generalitzacions, dependències, etc.
 - Com es relacionen les classes entre si?
 - Quin tipus de condició han de complir?

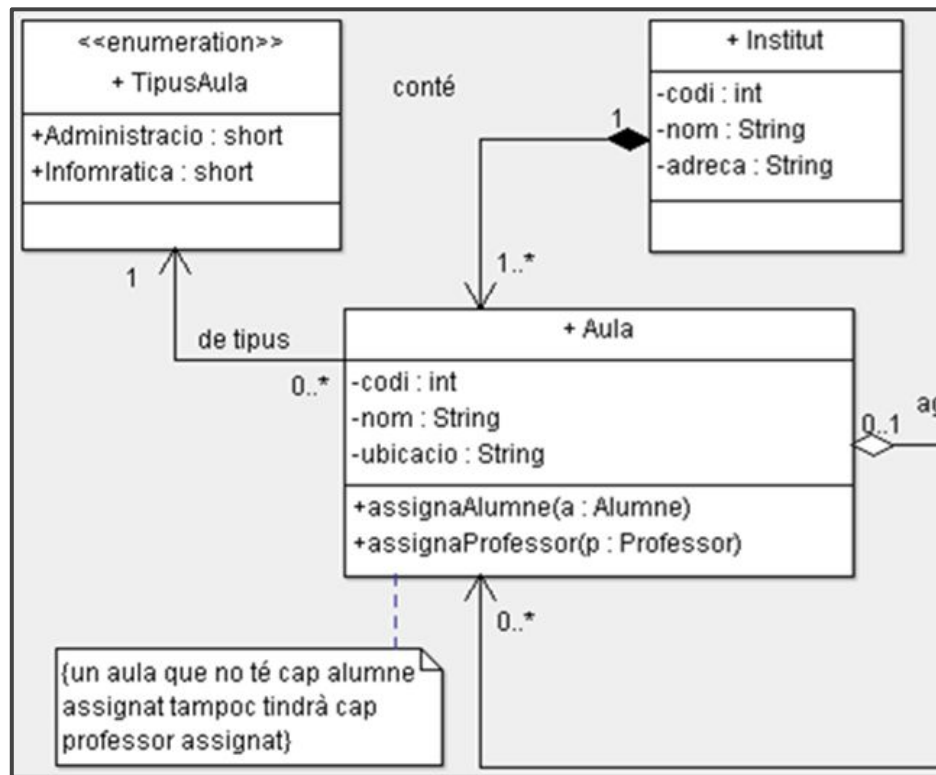
- Quins conceptes hem de saber de la programació orientat a objectes?
 - **Classe, atribut i mètode (operacions).**
 - **Visibilitat.**
 - **Objecte. Instanciació.**
 - Relacions. Herència, composició i agregació.
 - Classe associativa.
 - Interfícies.

Classe

- Es pot definir una classe com **l'element bàsic d'un model orientat a objectes**.
- És un contenidor que **agrupa un conjunt d'entitats que tenen les mateixes propietats, comportament, relacions i semàntica**.
- Les classes d'un UML es poden relacionar directament amb les classes d'un programa Java.

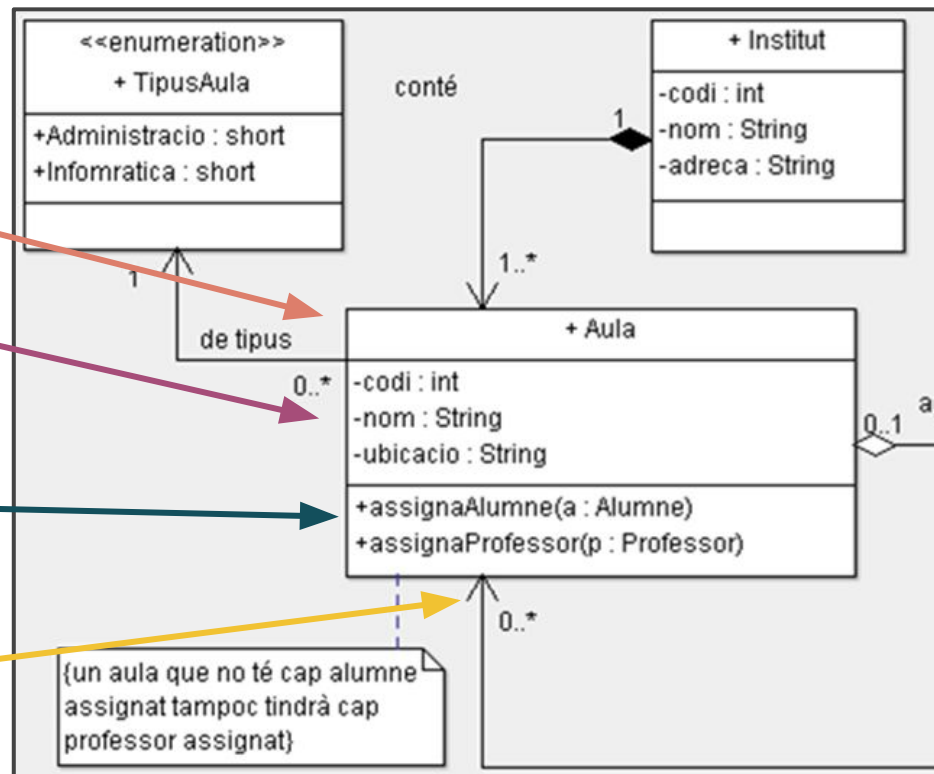
Classe

- Nom de classe
- Atributs
 - Visibilitat (encapsulació)
 - Nom
 - Tipus
- Mètodes
 - Visibilitat (encapsulació)
 - Nom
 - Tipus
- Relacions



Classe

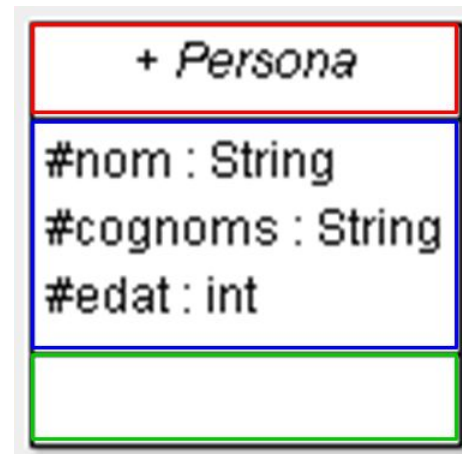
- Nom de classe
- Atributs
 - Visibilitat (encapsulació)
 - Nom
 - Tipus
- Mètodes
 - Visibilitat (encapsulació)
 - Nom
 - Tipus
- Relacions



Classe: composició

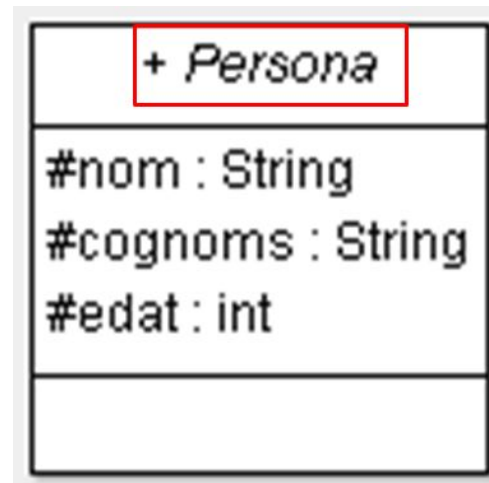
La classe es representa com una capsula dividida en tres seccions:

- La primera secció es reserva pel nom de la classe
- La segona secció es reserva per als atributs
- La tercera secció es reserva per als mètodes



Classe: el nom

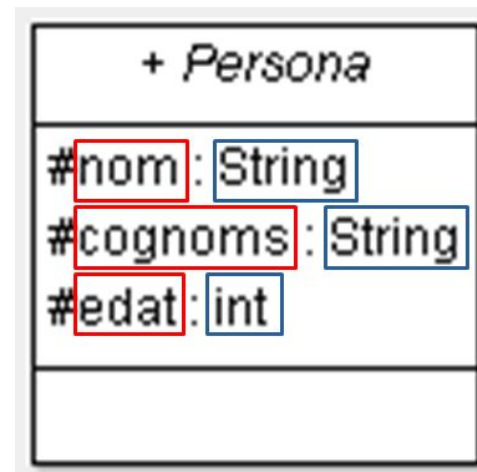
- El nom de la classe sempre comença per majúscules.
- Dues classes no es poden dir igual.
- No pot contenir espais.
- Si està en cursiva vol dir que la classe és **abstracta***



abstracta:* és una classe que no es pot instanciar, però sí que heretar.

Classe: els atributs

- Defineixen quines dades pot emmagatzemar la classe.
- Es defineixen amb el seu nom i el seu tipus.
- No poden contenir espais.



Atributs: tipus de dades

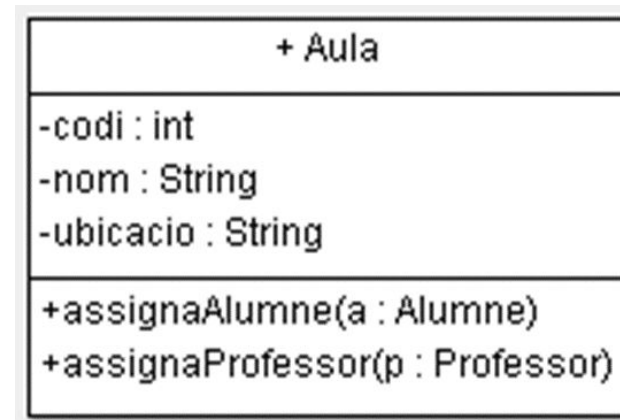
- El llenguatge UML té uns tipus estàndard definits, però permet definir-ne de nous.
- Els programes de modelatge (**com Modellio**) poden transformar els tipus UML en tipus Java per a generar-ne codi font.

- Exemple de tipus de dades en UML i equivalències amb Java:

	UML	Java
Tipus de dades	integer	short int long
	<u>string</u>	char String
	real	float double
	boolean	<u>boolean</u>

Classe: els mètodes

- Defineixen quin comportament tindrà la classe.
- Implementen les accions que es podran dur a terme sobre els atributs.
- Es defineixen amb el seu nom i els seus paràmetres que es defineixen com els atributs (nom : tipus).
- No poden contenir espais.

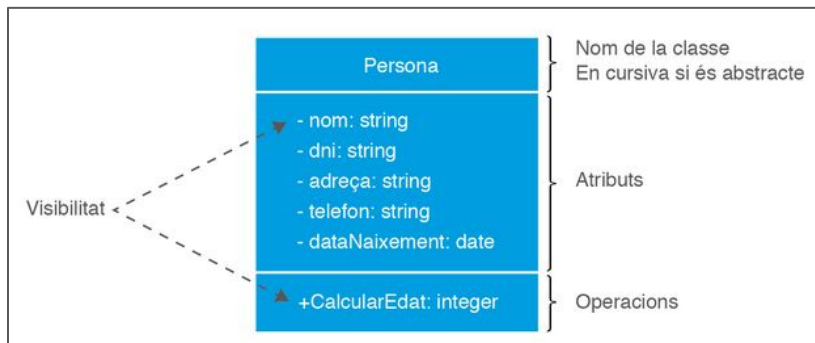


Classe: visibilitat o encapsulació

- S'aplica tant a classes com a mètodes.
- Indica si l'element serà visible (accessible) des d'una altra part del codi.
- Cada llenguatge pot implementar-ne tipus diferents.
- A Java en tenim quatre tipus (ordenats de menys a més restrictius).

Public	simbol +	Accesible desde tot el programa
Private	simbol -	Accesible només des de la classe
Protected	simbol #	Accesible des de les seves subclasses i, a Java, des de qualsevol classe en el mateix paquet
Package	simbol ~	Accesible des del paquet

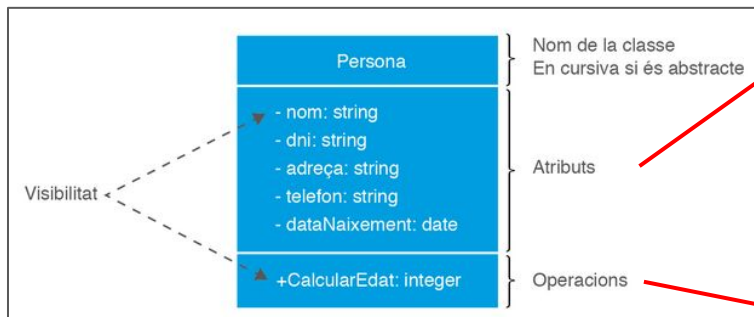




```
Class Persona{
{
    // atributs
    private String nom;
    private String dni;
    private String adreca;
    private String telefon;
    private date dataNaixement;
    // constructor
    public Persona(String nom, String dni, string adreca, string telefon, date dataNaixement) {
        this.nom = nom;
        this.dni = dni;
        this.adreca= adreca;
        this.telefon = telefon;
        this.dataNaixement = dataNaixement;
    }
    // mètodes o operacions
    public integer CalcularEdat() {
        Date dataActual = new Date();
        SimpleDateFormat format = new SimpleDateFormat("dd/MM/yyyy");
        String _dataActual = format.format(dataActual);
        String _dataNaixement = format.format(dataNaixement);

        String[] dataInici = _dataNaixement.split("/");
        String[] dataFi = _dataActual.split("/");

        int anys = Integer.parseInt(dataFi[2]) - Integer.parseInt(dataInici[2]);
        int mes = Integer.parseInt(dataFi[1]) - Integer.parseInt(dataInici[1]);
        if (mes < 0) {
            anys = anys - 1;
        } else if (mes == 0) {
            int dia = Integer.parseInt(dataFi[0]) - Integer.parseInt(dataInici[0]);
            if (dia > 0) anys = anys - 1;
        }
        return anys;
    }
}
```



Com podeu veure en l'exemple no indiquem els constructors*.

```
Class Persona{  
    // atributs  
    private String nom;  
    private String dni;  
    private String adreca;  
    private String telefon;  
    private date dataNaixement;  
    // constructor  
    public Persona(String nom, String dni, string adreca, string telefon, date dataNaixement) {  
        this.nom = nom;  
        this.dni = dni;  
        this.adreca= adreca;  
        this.telefon = telefon;  
        this.dataNaixement = dataNaixement;  
    }  
    // mètodes o operacions  
    public integer CalcularEdat() {  
        Date dataActual = new Date();  
        SimpleDateFormat format = new SimpleDateFormat("dd/MM/yyyy");  
        String _dataActual = formato.format(dataActual);  
        String _dataNaixement = formato.format(dataNaixement);  
  
        String[] dataInici = _dataNaixement.split("/");  
        String[] dataFi = _dataActual.split("/");  
  
        int anys = Integer.parseInt(dataFi[2]) - Integer.parseInt(dataInici[2]);  
        int mes = Integer.parseInt(dataFi[1]) - Integer.parseInt(dataInici[1]);  
        if (mes < 0) {  
            anys = anys - 1;  
        } else if (mes == 0) {  
            int dia = Integer.parseInt(dataFi[0]) - Integer.parseInt(dataInici[0]);  
            if (dia > 0) anys = anys - 1;  
        }  
        return anys;  
    }  
}
```

La figura de l'analista i els requisits.

Extracció de requisits:

- Primer pas: lectura.
- Segon pas: identificació de requisits.
- Tercer pas: identificació de classes, atributs i mètodes.
- Quart pas: construcció d'una classe.

La figura de l'analista i els requisits.

- L'analista és la persona encarregada de parlar amb les persones que intervindran en l'ús de l'aplicació (actors).
- L'objectiu és extreure un conjunt de requisits.
- Els requisits son tot allò que l'aplicació ha de complir perquè es doni per bona.
 - **Funcionals**
Requeriments que descriuen les funcionalitats que ha de satisfer l'aplicació a construir.
 - **No funcionals**
Requeriments relacionats amb la seguretat, el funcionament òptim, tolerància a errors, etc... és a dir, propis de la plataforma o arquitectura amb la qual es construeix l'aplicació.

EXERCICI 1. La figura de l'analista i els requisits

- A continuació es simularà que un analista fa arribar un escrit, detallant un problema que s'ha de resoldre.
- Seguint una sèrie de passos, es pot simplificar la tasca de generar diagrames de classes a partir d'un enunciat.
- Per simplicitat, es treballarà de moment amb una única classe.

Primer pas: la lectura

El concessionari de cotxes de segona mà "car2car" ens ha demanat una aplicació que sigui capaç d'emmagatzemar tota la informació referent a l'estoc de cotxes que tenen actualment.

En concret volen guardar per a cada cotxe quin és el seu model, marca i versió; també necessiten saber l'any de fabricació, el de matriculació, la data en la que es va adquirir el cotxe, a quin preu l'han comprat i a quin preu el volen vendre. Per últim necessiten saber a quin preu l'han acabat venent i en quina data l'han venut.

Tot i que sembla evident que és poca informació, ens han comentat que de moment no els hi fa falta i que, en tot cas, ja contactarien amb nosaltres per afegir-ne funcionalitats.

Segon pas: identificar els requisits

De l'enunciat que ha redactat l'analista, es pot separar el plantejament del problema dels requisits que ens passen per poder pensar el disseny:

*En concret **volen guardar** per a cada cotxe quin és el seu model, marca i versió; també **necessiten saber** l'any de fabricació, el de matriculació, la data en la que es va adquirir el cotxe, a quin preu l'han comprat i a quin preu el volen vendre.*

*Per últim **necessiten saber** a quin preu l'han acabat venent i en quina data l'han venut.*

Tercer pas: identificar classes i atributs

Amb la informació anterior com podem identificar les classes i atributs?

Tercer pas: identificar classes i atributs

Si ens fixem en el paràgraf on es detallen els requeriments, sembla senzill separar quin és l'element que l'empresa vol guardar (**la classe**) i quins són els atributs que fan falta:

En concret **volen guardar per a cada cotxe** quin és el seu model, marca i versió; també **necessiten saber** l'any de fabricació, el de matriculació, la data en la que es va adquirir el cotxe, a quin preu l'han comprat i a quin preu el volen vendre. Per últim **necessiten saber a quin preu l'han acabat venent i en quina data l'han venut**.

Quart pas: llistar i construir la classe

Amb la informació anterior com podem construir la classe?

Quart pas: llistar i construir la classe

Amb la informació anterior podem construir la classe d'aquesta forma:

➤ Classe:

- Cotxe

➤ Atributs:

- Model
- Marca
- Versió
- Any de fabricació
- Any de matriculació
- Data de compra
- Preu de compra
- Preu de venda public
- Preu de venda final
- Data de venda



EXERCICI 1.2 Ampliació: un canvi inesperat (o no) en els requisits

Un altre analista redacta el següent enunciat després d'una conversa amb el client:

Bona tarda,

He estat parlant amb el client i, tal i com veiem venir, s'han quedat curts amb les especificacions que ens van passar. Diuen que no poden saber si un cotxe és blau, si està tot ratllat o si en té quatre o sis portes. A més volen saber si un cotxe s'ha pogut provar. Creus que això canvia el model que vam plantejar al començament?

Salutacions,



Ampliació: extracció (de nou) de requisits.

De la mateixa forma que vam procedir al començament, anem a veure si es poden extreure els requeriments de forma senzilla...

Ampliació: extracció (de nou) de requisits

De la mateixa forma que vam procedir al començament, anem a veure si es poden extreure els requeriments de forma senzilla:

Bona tarda,

*He estat parlant amb el client i, tal i com veiem venir, s'han quedat curts amb les especificacions que ens van passar. Diuen que **no poden saber si un cotxe és blau, si està tot ratllat o si en té quatre o sis portes. A més volen saber si un cotxe s'ha pogut provar.** Creus que això canvia el model que vam plantejar al començament?*

Salutacions,

Ampliació: identificar elements

Un truc senzill, però **no infal·lible**, per a determinar els elements que necessitem pel nostre model pot ser:

- substantius (cotxe, bicicleta, avió, ...) → classes
- adjectius (blau, car, ratllat, vell, ...) → atributs
- verbs (provar, reparar, ...) → mètodes

Bona tarda,

He estat parlant amb el client i, tal i com veiem venir, s'han quedat curts amb les especificacions que ens van passar. Diuen que **no poden saber si un cotxe és blau, si està tot ratllat o si en té quatre o sis portes**. A més **volen saber si un cotxe s'ha pogut provar**. Creus que això canvia el model que vam plantejar al començament?

Salutacions,

Ampliació: afegir nous elements a la classe

➤ Classe:

- Cotxe

➤ Atributs:

- Model
- Marca
- Versió
- Any de fabricació
- Any de matriculació
- Data de compra
- Preu de compra
- Preu de venda public
- Preu de venda final
- Data de venda

- Preu de venda public
- Preu de venda final
- Data de venda
- Color
- SI està ratllat
- Número de portes

➤ Mètodes

- Provar el cotxe



EXERCICI 2. Crea el diagrama de classes d'aquest requeriment

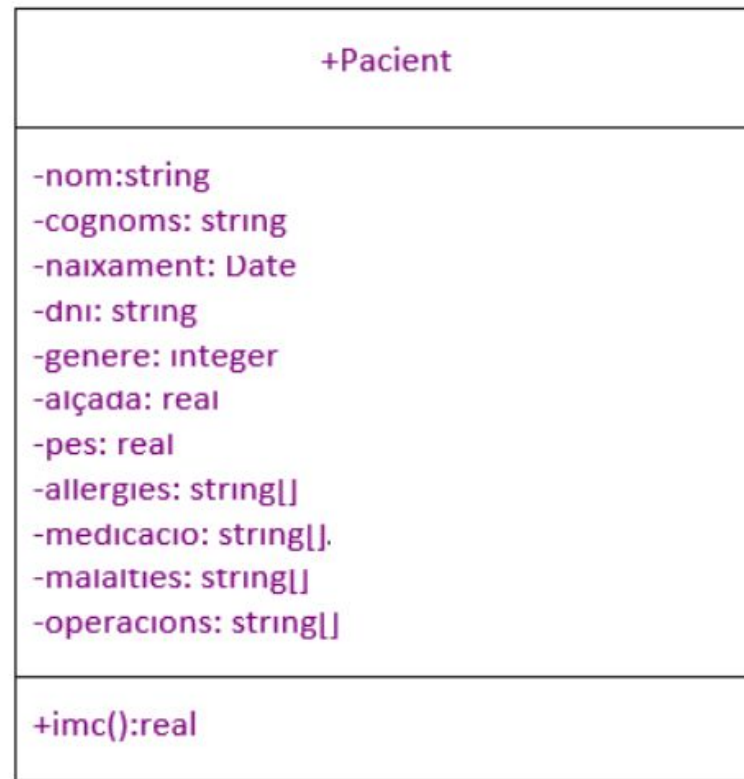
Un metge necessita una aplicació per mantenir les dades dels pacients, en concret necessita guardar el següent: nom; cognoms; data de naixement; DNI; gènere; alçada; pes; al·lèrgies; nom dels medicaments que pren actualment; quines malalties cròniques pateix el pacient; operacions quirúrgiques a les quals s'ha exposat el pacient. Per últim ens demana que l'aplicació calculi de forma automàtica l'índex de massa corporal del pacient (IMC).

Crea el diagrama de classes d'aquest requeriment

*Un metge necessita una aplicació per mantenir les dades dels **pacients**, en concret necessita guardar el següent: **nom; cognoms; data de naixement; DNI; gènere; alçada; pes; al·lèrgies; nom dels medicaments que pren actualment; quines malalties cròniques pateix el pacient; operacions quirúrgiques a les quals s'ha exposat el pacient.** Per últim ens demana que l'aplicació **calculi de forma automàtica l'índex de massa corporal del pacient (IMC).***

Requeriments

*Un metge necessita una aplicació per mantenir les dades dels **pacients**, en concret necessita guardar el següent: **nom; cognoms; data de naixement; DNI; gènere; alçada; pes; al·lèrgies; nom dels medicaments que pren actualment; quines malalties cròniques pateix el pacient; operacions quirúrgiques a les quals s'ha exposat el pacient. Per últim ens demana que l'aplicació **calculi de forma automàtica l'índex de massa corporal del pacient (IMC).*****



EXERCICI 3. Crea el diagrama de classes d'aquests requeriments

En Pere ha obert una nova fruiteria al mercat i té molta curiositat per saber quina quantitat de fruita està venent, quin és el preu de cost de tota la fruita que ha venut i quants diners porta guanyats en total. De moment no li interessa fer un desglossament dels imports per tipus de fruita, però sí que li agradaria calcular de forma fàcil quin ha sigut el seu marge de benefici.

Crea el diagrama de classes d'aquests requeriments

*En Pere ha obert una nova **fruiteria** al mercat i té molta curiositat per saber quina **quantitat** de fruita està venent, quin és el **preu de cost** de tota la fruita que ha venut i **quants diners porta guanyats en total**. De moment no li interessa fer un desglossament dels imports per tipus de fruita, però sí que li agradaria **calcular de forma fàcil quin ha sigut el seu marge de benefici**.*

Requeriments

*En Pere ha obert una nova **fruiteria** al mercat i té molta curiositat per saber quina **quantitat** de fruita està venent, quin és el **preu de cost** de tota la fruita que ha venut i **quants diners porta guanyats en total**. De moment no li interessa fer un desglossament dels imports per tipus de fruita, però sí que li agradaria **calcular de forma fàcil quin ha sigut el seu marge de benefici**.*



Relacions

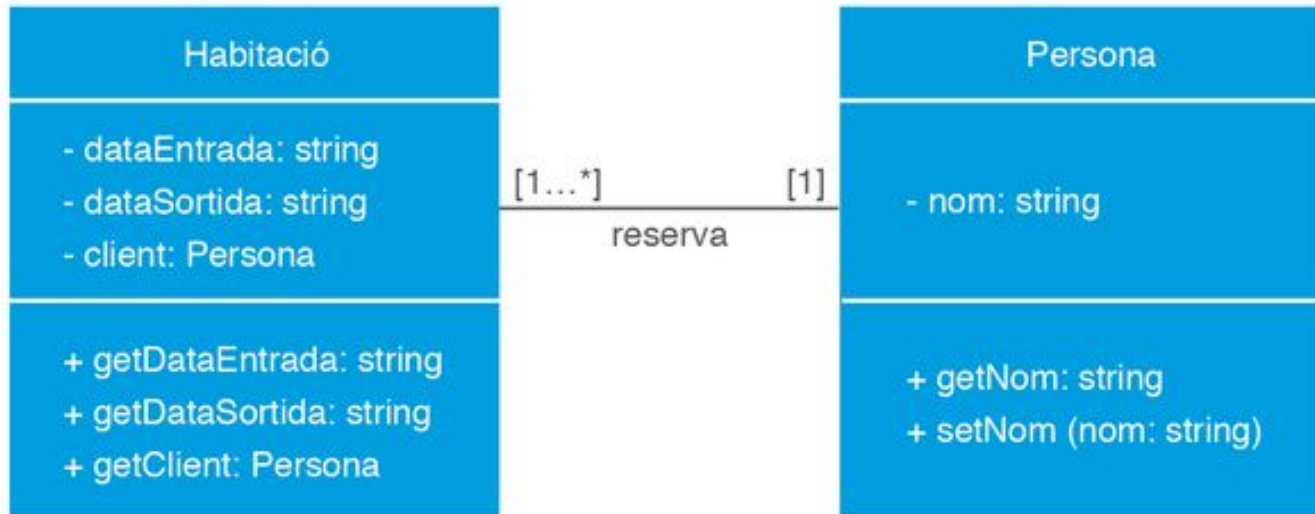
- Les **classes es relacionen unes amb les altres**.
- Les relacions que existeixen entre les diferents classes s'anomenen **associacions** de forma genèrica.
- Quan es representa una associació entre classes, també s'indica la **multiplicitat i el rol de navegació** d'aquesta relació.
- La **multiplicitat** (que es representa amb una valors **[min...max]**) indica el nombre mínim i el màxim d'enllaços possibles que es poden donar en la relació.

Relacions

Hi ha 6 tipus de relacions entre classes:

1. Relació d'associació
2. Relació d'associació d'agregació
3. Relació d'associació de composició
4. Relació de dependència
5. Relació de generalització
6. Classe associativa

Relacions



Relació d'associació

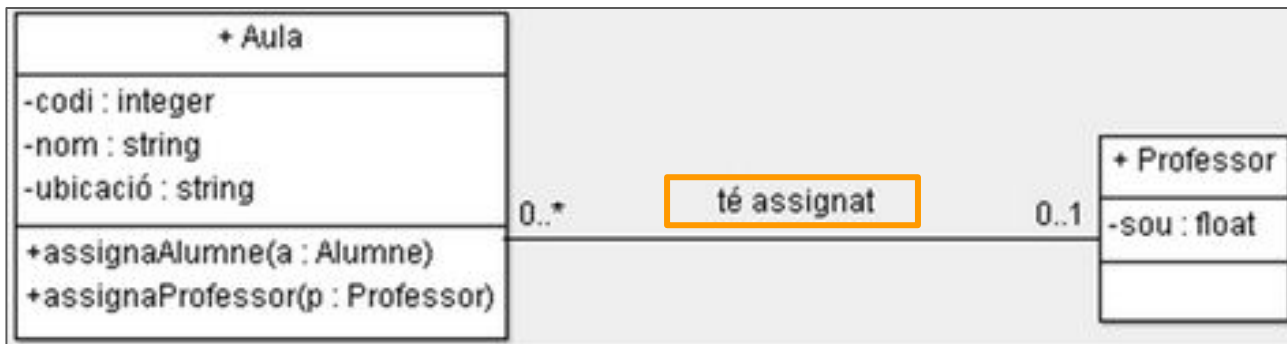
- Es representa mitjançant **una línia contínua sense fletxes ni cap altre símbol als extrems.**
- La relació d'associació és **bidireccional**. (Una línia amb una fletxa a cada extrem és equivalent a la línia sense símbols als extrems.)
- Representa que **les dues classes col·laboren entre si.**



Relació d'associació

Rol de navegació

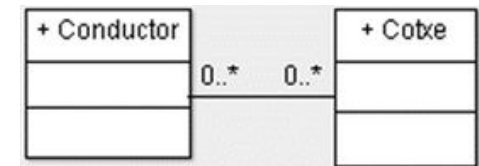
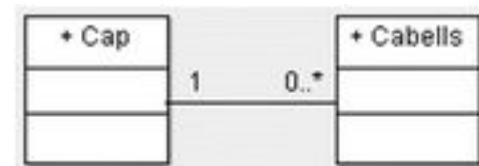
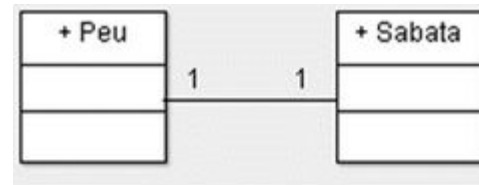
- Defineix com hem d'interpretar o llegir l'associació.
- Ho indiquem sobre la fletxa, **fent ús d'un verb**.



Relació d'associació: Cardinalitat

La cardinalitat en una relació entre entitats i és el nombre de vegades que una entitat apareix associada a una altra entitat.

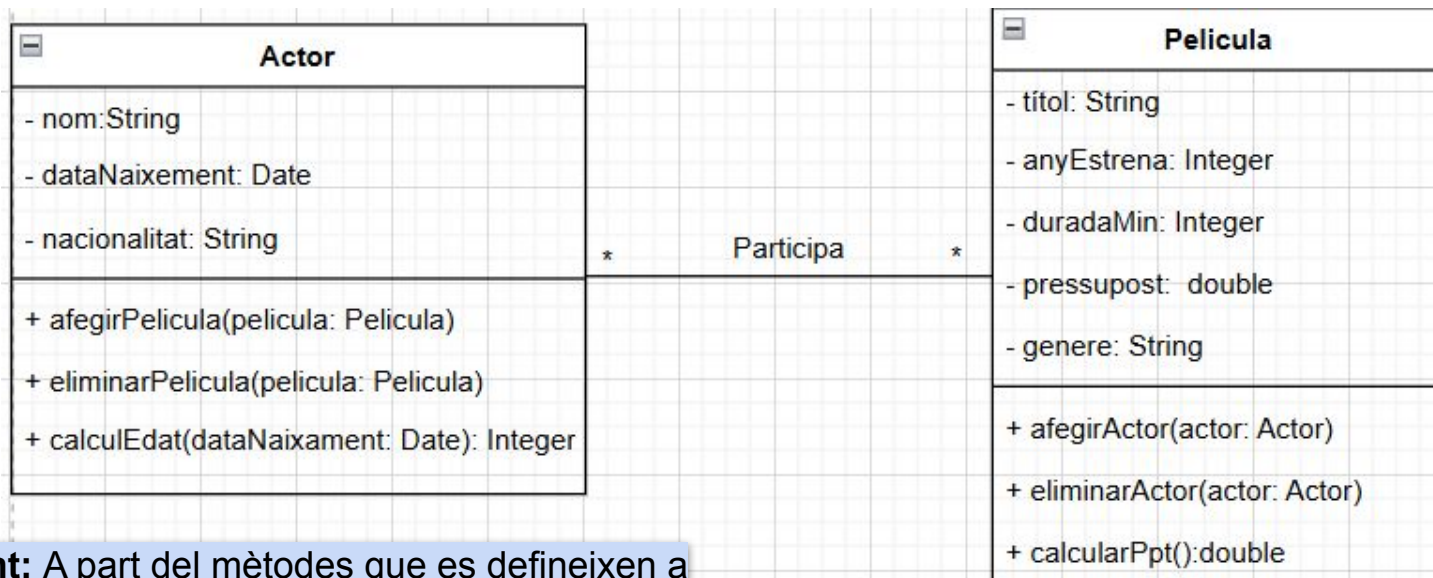
- Relació 1 a 1:
 - Apareix com un “1” a cada extrem de la relació.
 - Les instàncies de cada entitat participen cadascuna d'elles una vegada en la relació
- Relació 1 a molts:
 - En un dels extrems hi ha un “1” i a l'altre un rang de valors o un sol valor més gran que 1.
 - Exemples:
 - 0..1, de 0 a 1.
 - 0..*, de 0 fins a infinit
- Relació molts a molts:
 - És una generalització de la relació 1 a molts, doncs a cada extrem apareix un rang de valors o un sol valor més gran que 1.



Tenim nous requeriments ...

Una productora de cinema necessita un sistema per gestionar la informació sobre pel·lícules i actors. De les pel·lícules en vol emmagatzemar el títol, l'any de llançament, la durada en minuts i el gènere, i s'ha de poder afegir i eliminar actors del repartiment, calcular el pressupost total. Dels actors en vol saber el nom, la data de naixement i la nacionalitat, i s'ha de poder afegir i eliminar pel·lícules en les quals han participat i també es vol calcular l'edat actual. Un actor pot haver participar en múltiples pel·lícules i una pel·lícula pot tenir diversos actors.

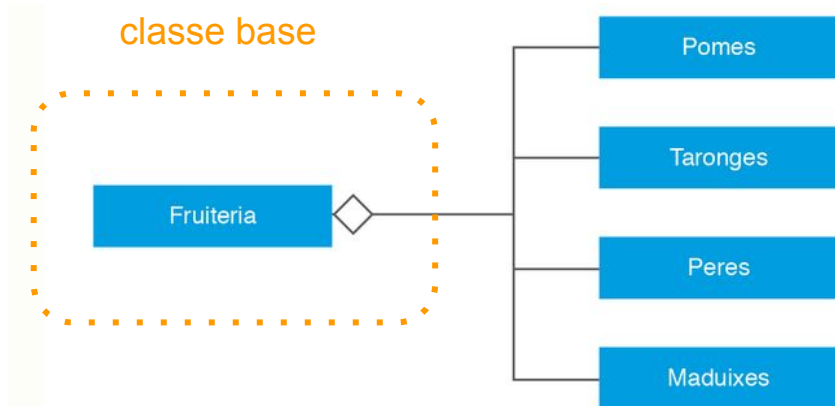
Una productora de cinema necessita un sistema per gestionar la informació sobre pel·lícules i actors. De les **pel·lícules** en vol emmagatzemar **el títol, l'any de llançament, la durada en minuts i el gènere**, i s'ha de poder **afegir i eliminar actors del repartiment, calcular el pressupost total**. Dels **actors** en vol saber **el nom, la data de naixement i la nacionalitat**, i s'ha de poder **afegir i eliminar pel·lícules en les quals han participat i també es vol calcular l'edat actual**. Un actor pot haver participar en múltiples pel·lícules i una pel·lícula pot tenir diversos actors.



Important: A part del mètodes que es defineixen a l'enunciat s'ha de considerar incloure **els constructors** i **els mètodes accessors (getters/setters)**

Relació d'associació d'agregació

- Es representa mitjançant una línia contínua que finalitza en un dels extrems per un rombe sense omplir.
- Es fa servir per representar que una classe (base) està formada per una o més classes.
- La relació indica que la classe base necessita de les classes incloses per poder existir i fer les seves funcionalitats.



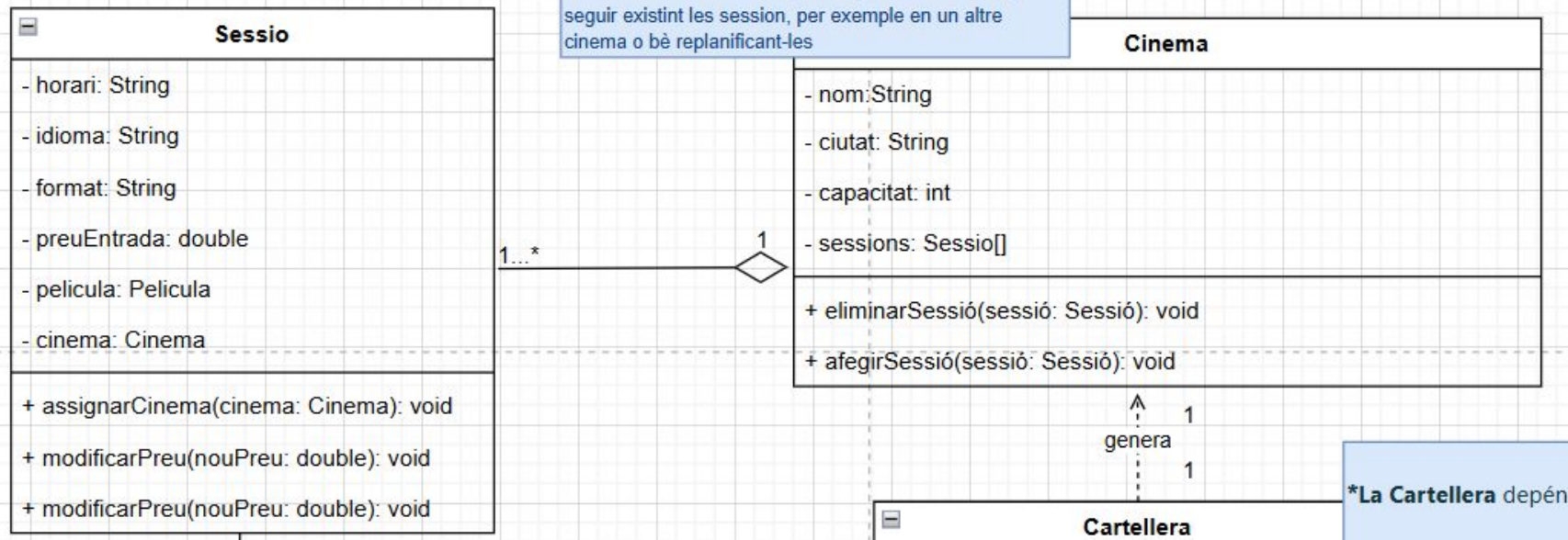
En aquest exemple on hi ha una relació de **tot-part** (on les fruites son part de la fruiteria), si la fruiteria deixa d'existir, les fruites continuen existint. (es podria llegir com **està format per**)

Un nou requeriment...

A més, es vol gestionar informació sobre les sessions de projecció. Cada sessió representa la projecció d'una pel·lícula en un cinema en un horari específic i es vol emmagatzemar horari de la projecció, l'idioma en què es projecta, el format de la projecció (per exemple, 2D, 3D) i finalment el preu de l'entrada. I del cinema es vol saber el nom, la ciutat on es troba i la capacitat de la sala, també volen saber quines pel·lícules es projecten.

Un cinema pot tenir múltiples sessions de projecció, i cada sessió ha d'estar associada a un únic cinema, ja que no té sentit sense un cinema.

*El cinema és el cas base, si desapareix el cinema poden seguir existint les sessions, per exemple en un altre cinema o bé replanificant-les

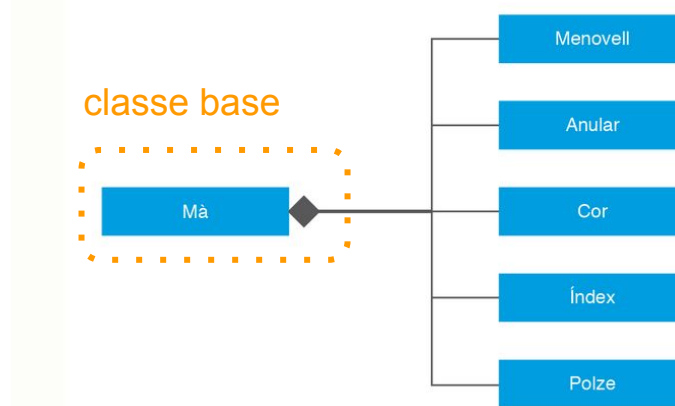


*La Cartellera depèn

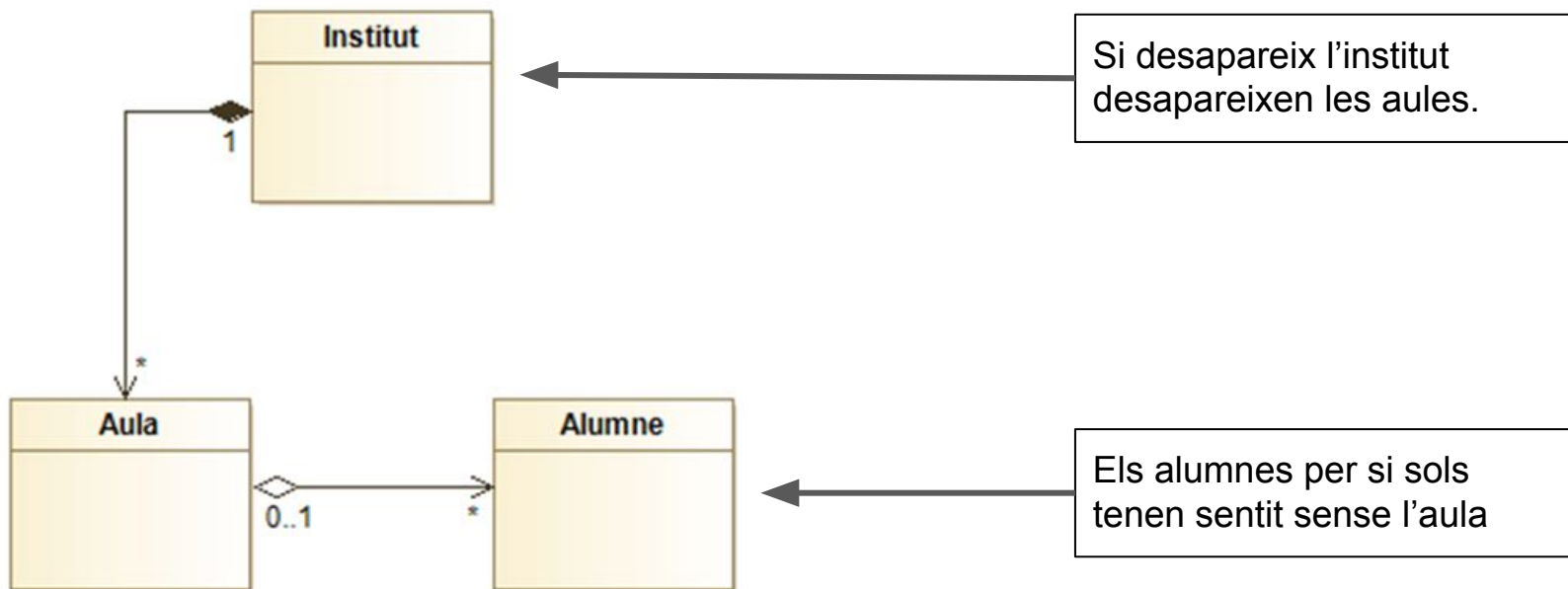
Es defineix una **relació d'agregació** entre **Cinema** i **Sessió**, ja que un cinema conté sessions, però aquestes sessions poden existir independentment de qualsevol altra sessió d'aquell cinema. La relació **Cinema - Sessió** té sentit com a **agregació**, perquè una Sessió **pot ser traslladada d'un Cinema a un altre** en cas necessari. **No hi ha dependència forta**: si s'elimina un Cinema, les Sessions poden ser reassignades.

Relació d'associació de composició

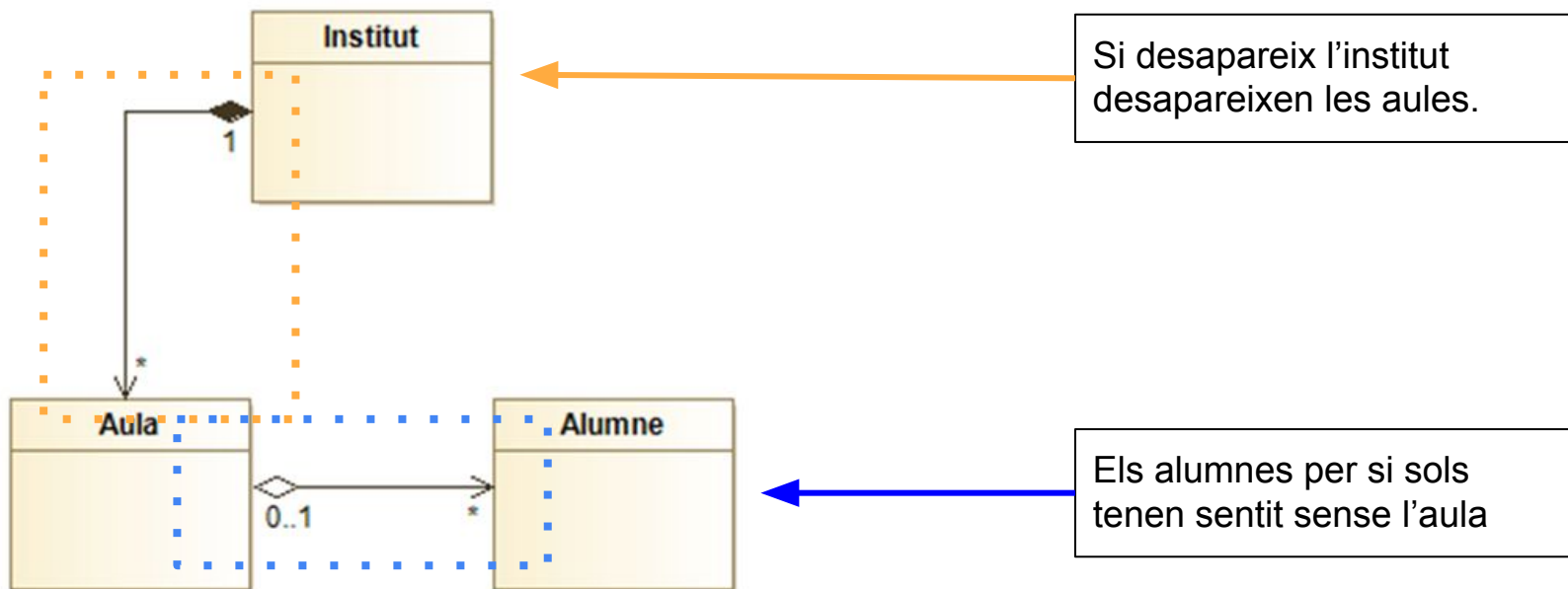
- Es representa mitjançant **una línia contínua que finalitza en un dels extrems per un rombe omplert**. També es podria llegir com **està format per** i mostra una relació més forta que una agregació.
- També serveix per **representar una classe (base) que està composta d'altres classes**.
- Hi ha una dependència d'existència entre la classe base i les classes que hi està inclòs. És a dir, **si deixa d'existir la classe base, deixarà d'existir també les classes incloses**.



Agregació vs Composició

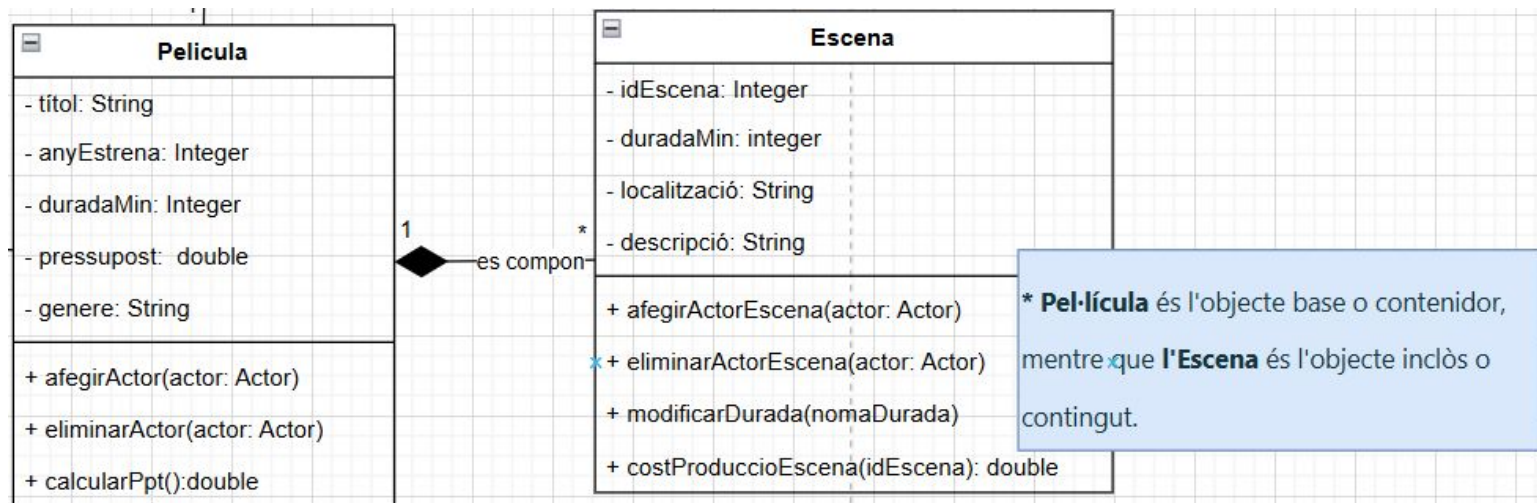


Agregació vs Composició



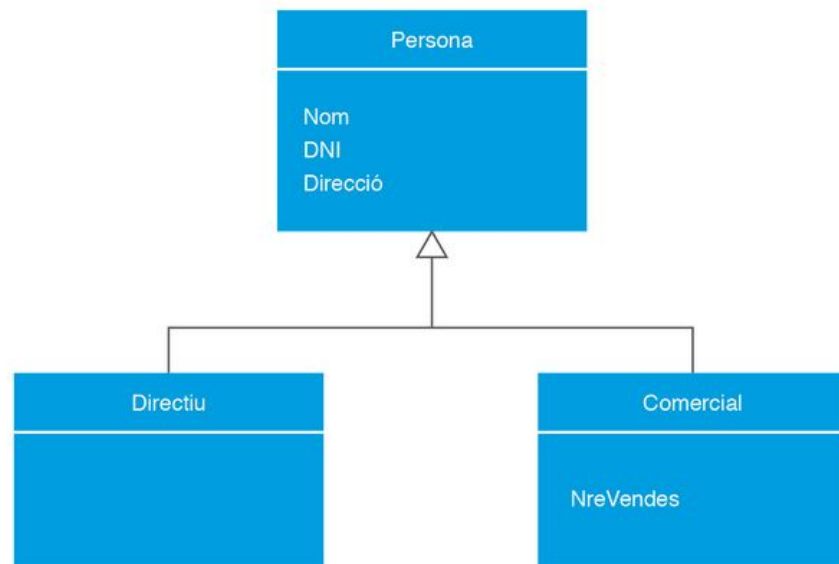
Seguim amb més canvis...

Ara volem afegir les Escenes de la pel·lícula. De l'escena es necessita atributs que permetin identificar-la de manera única, descriure la seva durada, especificar el lloc on es desenvolupa i proporcionar una breu descripció del seu contingut. Es vol afegir i eliminar elements que participen en ella, canviar la seva durada i saber el cost de producció. La relació amb la pel·lícula és especialment important, ja que una pel·lícula està formada per diverses escenes i una escena pertany a una única pel·lícula.



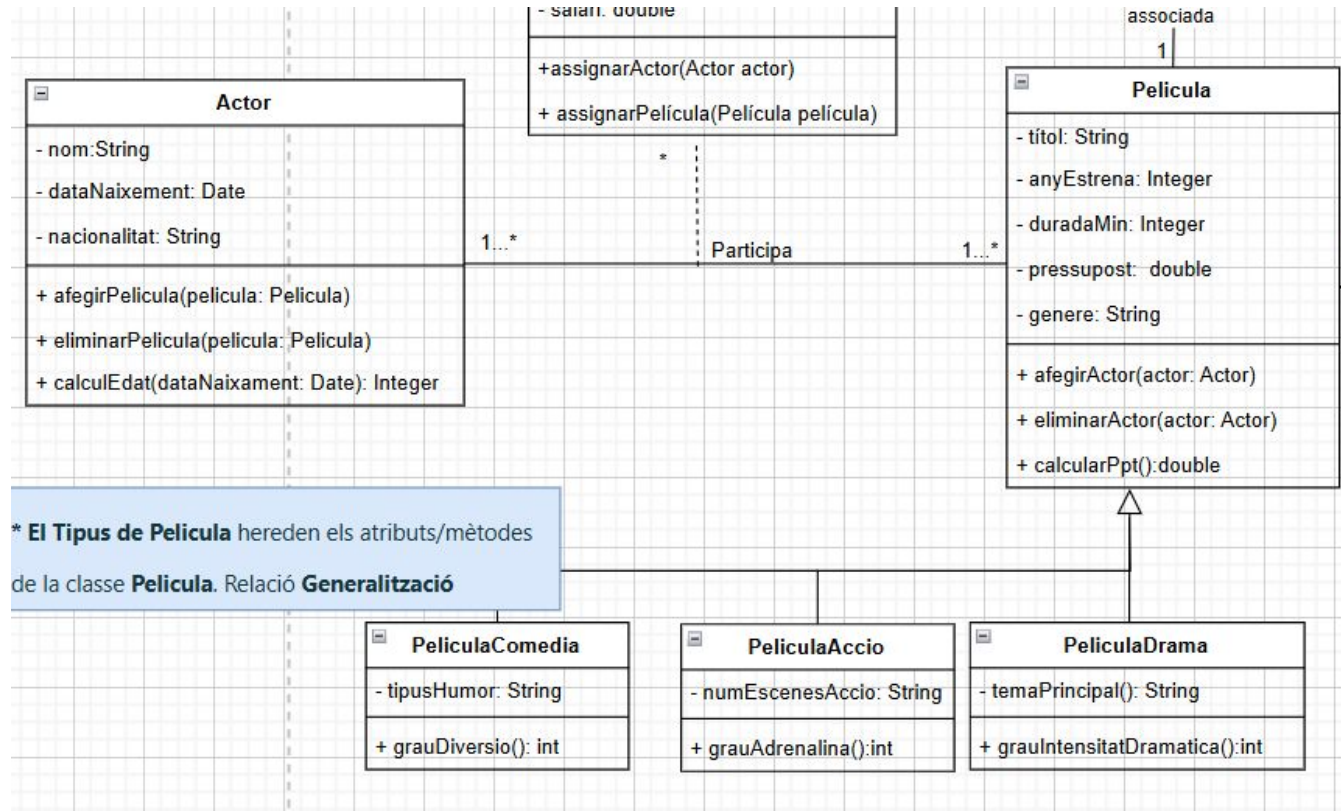
Relació de generalització (herència)

- la relació d'herència entre classes es representa mitjançant una línia contínua que finalitza amb un triangle.
- Les classes **filles o subclasses** hereten els **atributs, els mètodes i comportament** de la classe mare.
- Una classe és anomenada classe mare (o superclasse). L'altra (o les altres) són les anomenades classes filles o subclasses, que hereten els atributs i els mètodes i comportament de la classe mare.



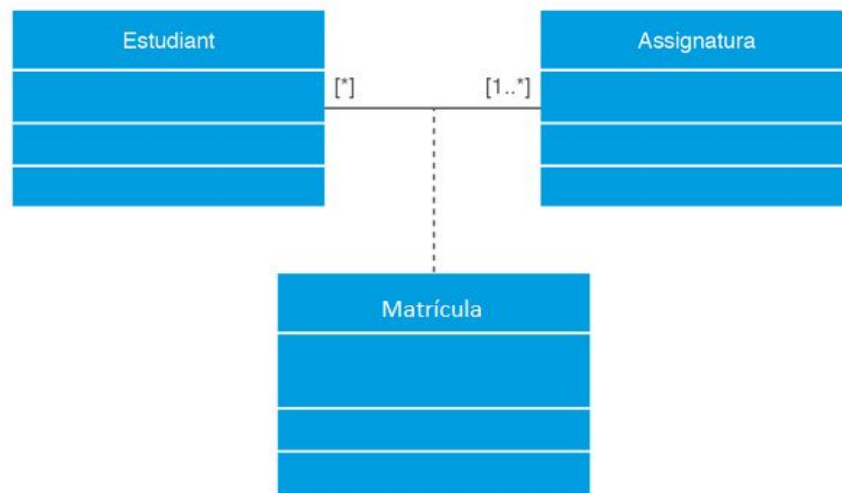
La classe Película es classifica en tres tipus: PelículaDrama, PelículaComèdia i PelículaAcció. La PelículaDrama afegeix l'atribut temaPrincipal (per exemple, "amor" o "guerra") i es vol saber el grau d'intensitat dramàtica. La PelículaComèdia inclou l'atribut tipusHumor (per exemple, "negre" o "absurd") i es vol saber el grau de diversió. Finalment, la PelículaAcció afegeix l'atribut nombreEscenesAcció i es vol calcular el grau d'adrenalina.

Diagrama de classes:



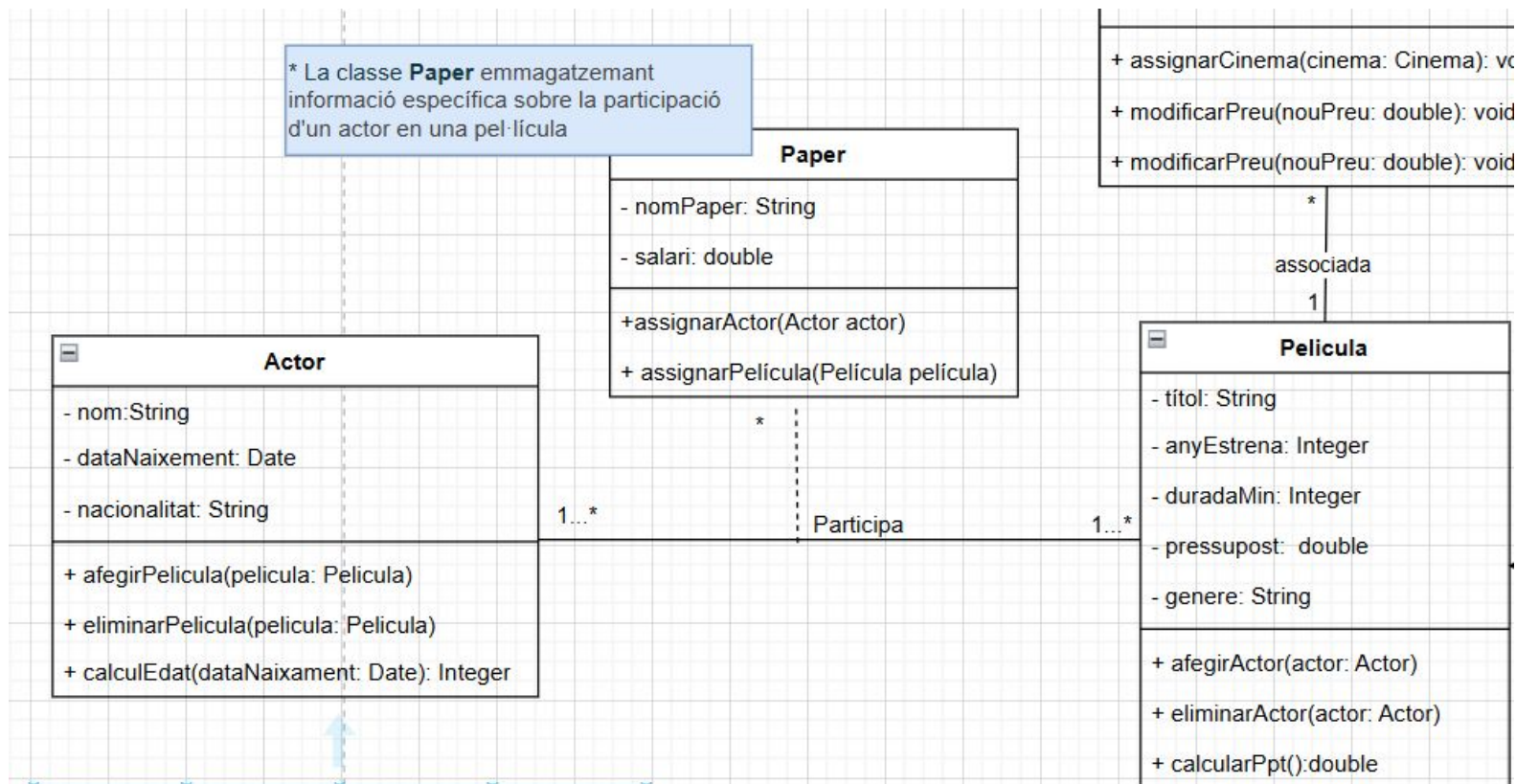
Relació de classe associativa

- Es representa mitjançant **una nova classe unida a la línia d'associació entre dues classes per mitjà d'una línia discontinua**.
- Es fa servir quan **la relació entre les classes té propietats o mètodes propis**.



Ara es vol emmagatzemar informació específica del paper de l'actor en relació a la pel·lícula i es vol saber el nom del paper i el salari i es vol poder assignar un actor a un paper i una pel·lícula a un paper. Un actor pot tenir múltiples papers i una pel·lícula pot tenir múltiples papers, però cada paper està associat a un únic actor i una única pel·lícula

Diagrama de classes:



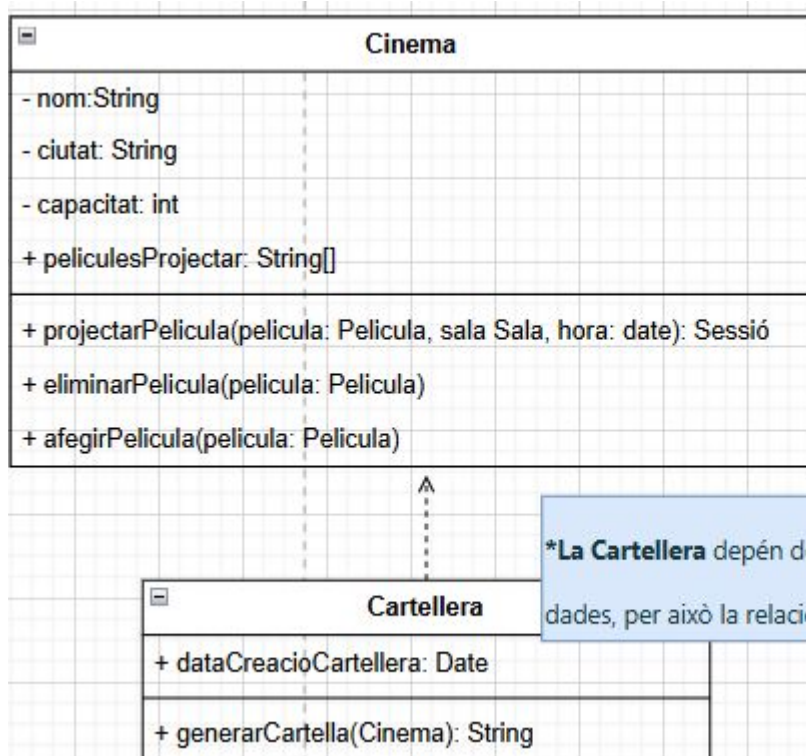
Relació de dependència

- Es representa mitjançant una línia discontinua amb una fletxa en un dels extrems.
- L'objecte del qual surt la fletxa es considera un objecte dependent. És una relació unidireccional:
 - l'objecte depenent té informació de l'objecte del qual depèn però a l'inrevés no.



S'ha identificat la necessitat d'incloure una nova funcionalitat: generar una cartellera per a un cinema específic, que mostri les pel·lícules que s'estan projectant actualment en aquest cinema, juntament amb informació rellevant com el títol, l'horari de projecció i la durada de cada pel·lícula. Al cinema s'afegiran les funcionalitats, d'afegir i eliminar pel·lícula.

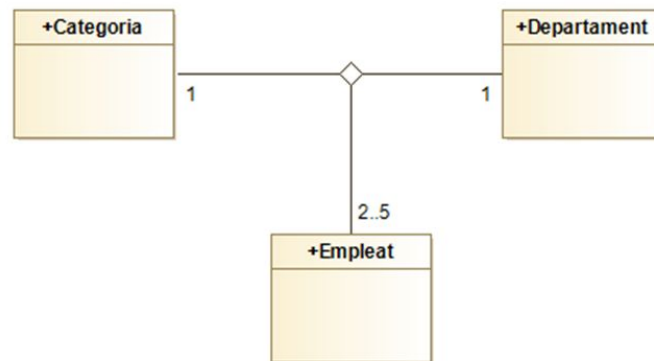
La classe Cartellera guarda la data de generació del cartell i res més, i sinó que dependrà de la classe Cinema per obtenir les dades necessàries per generar la cartellera.



***La Cartellera** depén del cinema per obtenir les dades, per això la relació de dependència.

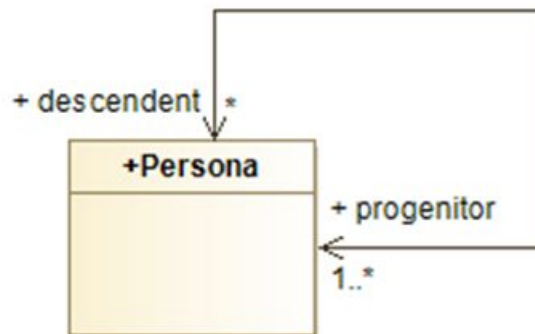
Relació Ternària

- Aquelles relacions que involucren tres classes alhora com a origen o destí de les mateixes.
- L'exemple de la dreta diu:
 - Un empleat d'una categoria pertany a un departament.
 - Un empleat d'un departament tindrà una categoria.
 - Una categoria i un departament tindrà de dos a cinc empleats.
 - Un empleat pot tindre més d'una categoria dins de diferents departaments.
 - Un empleat pot formar part de més d'un departaments, amb diferents categories.
- **Les relacions ternàries poden comportar complexitat en el disseny i en la implementació. En molts casos es poden substituir amb dues relacions d'associació.**



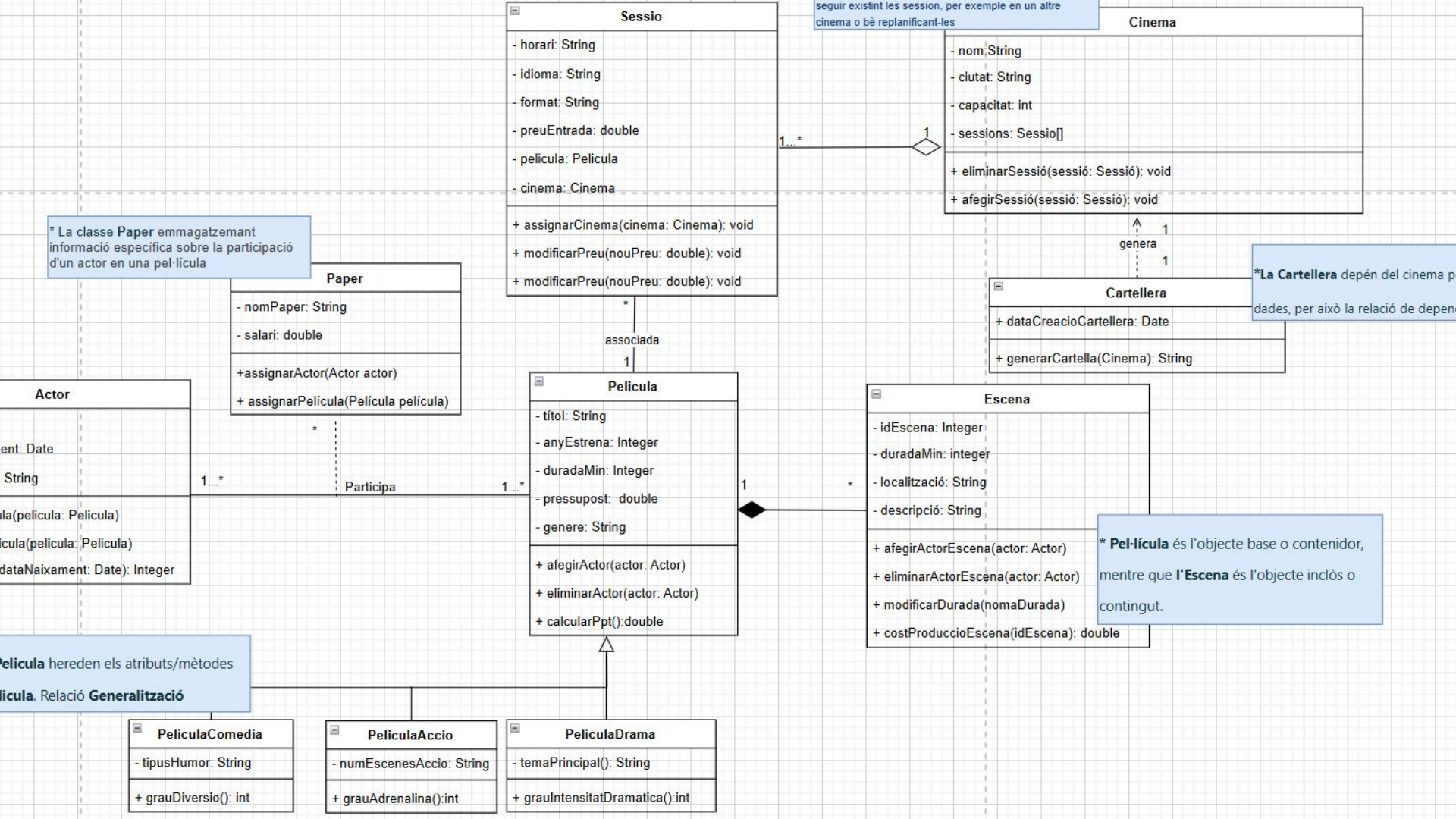
Relació unària (recursiva)

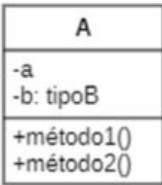
- Són aquelles relacions que comencen i acaben a la mateixa classe.
- Es representa com una fletxa que s'origina i acaba a la mateixa entitat.



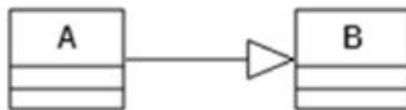
S'ha identificat la necessitat d'incloure una nova funcionalitat: gestionar sessions de projecció en un cinema, que mostri les sessions disponibles per a cada pel·lícula, incloent sessions alternatives (per exemple, sessions en diferents idiomes o formats). Es vol emmagatzemar informació sobre una sessió de projecció, com l'horari, l'idioma i el format (per exemple, 2D, 3D) i gestionar sessions alternatives, ja que una sessió pot tenir altres sessions relacionades (per exemple, una sessió en 3D pot tenir una sessió alternativa en 2D).

A més, s'ha de tenir en compte que una Sessió pot tenir zero o més sessions alternatives i que cada Sessió està associada a una Pel·lícula i a un Cinema. A més el sistema també ha de permetre afegir, eliminar i consultar sessions alternatives a una sessió.



Nombre	Notación	Descripción
Clase		Descripción de la estructura y comportamiento de un conjunto de objetos.
Clase abstracta		Clases que no pueden ser instanciadas. El nombre de la clase está en cursiva.
Asociación		Relaciones entre clases.
Composición		Relaciones partes-todo dependientes (A es parte de B; si B es eliminado, las instancias relacionadas de A son eliminadas).
Agregación		Relaciones partes-todo (A es parte de B; si B es eliminado, las instancias relacionadas de A no son eliminadas).

Generalización



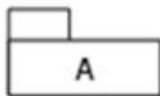
Relación de herencia (A hereda de B).

Interface



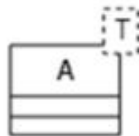
Elemento que declara un conjunto de métodos sin su implementación. Especifica un servicio proporcionado por una clase (A implementa B).

Paquetes



Elemento organizativo de diagramas, puede agrupar elementos de cualquier tipo si están relacionados semánticamente.

Clase genérica



Plantilla de clase que se puede parametrizar con uno o más tipos de datos.

Fuente: adaptado de Seidl *et al.* (2015).

- Els **objectes** representen formes concretes de les classes i són referits com a **instàncies**. Un ***objecte té una identitat única*** i una **sèrie de característiques que el descriuen amb més detall**.
- Aquest generalment interactua i es comunica amb altres objectes. Les relacions entre objectes es denominen enllaços.
- Les **característiques** d'un objecte inclouen les seves característiques **estructurals (atributs)** i el seu **comportament (mètodes)**.
- Valors concrets s'assignen als atributs en el diagrama d'objectes; les **operacions usualment no s'indiquen**.

El primer compartiment conté informació en la forma nombreObjeto: Classe



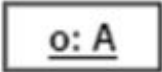
La segona notació vàlida és el nom de l'objecte sense col·locar el seu tipus (figura A2.2[b]).



Figura A2.3. Notación de objetos relacionados y con valores de sus atributos

Resum:

Tabla A2.2. Conceptos principales de los diagramas de objetos UML

Nombre	Notación	Descripción
Objeto		Instancia de una clase
Enlace		Relaciones entre objetos

Fuente: adaptado de Seidl *et al.* (2015).